

Real Time Operating System

Sonakshi Tripathi

Computer Science and Engineering
Lovely Professional University
Jalandhar, India

Anurag Yadav

Computer Science and Engineering
Lovely Professional University
Jalandhar, India

Shivali V.

Computer Science and Engineering
Lovely Professional University
Jalandhar, India

Abstract— An RTOS is a unique type of operating system created to manage tasks that require prompt and timely execution. What makes an RTOS special is its ability to respond to events in the environment within a very small, set amount of time so that specific critical deadlines can be met for those applications where timing matters most. They are deployed in areas such as embedded systems, industrial automation, automotive industries, medical devices and aerospace where guaranteed execution is essential. RTOS must have basic features like multi-tasking, preemptive scheduling, low latency and determinism along with more advanced interrupt handling as well as efficient memory management. That is, on general-purpose operating systems the rule here has been "flexibility" at any price, whereas around real-time there have been a semantic shift toward predictable response times. Moreover, the operating system code is tailored and optimized for tiny hardware-constrained environments; It utilizes unique scheduling algorithms of pre-emptive time-based to operate in these devices. RTOS can find use in a plethora of industries providing the benefit to offer more dependable robust and timely systems thus making them indispensable for cases which require predictable and consistent performance.

Keywords— Real-Time Operating System (RTOS), Deadline-driven Computing, Critical Task Scheduling, Preemptive Scheduling, Time Constraints, Embedded Systems, Industrial Automation, Aerospace Systems, Automotive Systems, Medical Devices, Low Latency, Deterministic Behavior, Interrupt Handling, Memory Management, Predictable Response Times, Task Execution Deadlines, Hardware-Constrained Environments, Real-Time System Design, Task Prioritization, Scheduling Algorithms, Multi-tasking, Embedded Real-Time Systems, Critical Applications, System

Optimization, Preemptive Time-based Scheduling, Robust and Dependable Systems, Hard Real-Time Systems, Software Predictability, RTOS in Aerospace, RTOS in Industrial Applications.

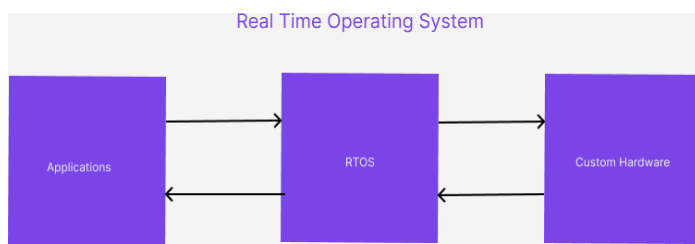
I. INTRODUCTION

A Real Time Operating System (RTOS) is an operating system developed for real-time embedded applications evolved around processors or controllers. It allows priorities to be changed instantly and data to be processed rapidly enough that the results may be used in response to another process taking place at the same time, as in transaction processing [1]. Although GPOS is designed for versatility and handling multiple tasks across various applications, the main purpose of an RTOS is to guarantee that essential tasks are finished within specified time limits. Therefore, timely and predictable reactions to external events are the main priority for the application, where delays can lead to disastrous outcomes.

They are frequently utilized in sectors with more demanding deadlines for guaranteed implementation, such as embedded systems, industrial automation, automotive technologies, medical equipment, and aerospace. In such settings, failing to meet deadlines may lead to system breakdowns, safety hazards, or data loss; thus, the function of an RTOS is essential for ensuring reliability and safety.

Additional notable features of an RTOS consist of multitasking, preemptive scheduling, minimal latency, predictable behavior, and efficient memory management. These features, consequently, provide the RTOS with the ability to multiplex in a multi-tasking setting, ensuring that high-priority tasks are performed first. Moreover, RTOSs offer advanced mechanisms that facilitate the handling of interrupts, thereby enabling a timely response to system events. System resources are preserved and effectively used by employing a hardware-constrained environment optimized for efficient operation on systems with limited processing and memory.

The need for implementing RTOSs comes from the complexity of contemporary applications in areas like health care, transportation, and manufacturing, among others. RTOSs have gained significance in ensuring the reliability of mission-critical systems, keeping pace with technological progress to fulfill realistic real-time demands. In the following text, we aim to explore the fundamental concepts of RTOS, its characteristics, and its uses in determining the significance of real-time processing amid the current technological changes.



II. DESIGN

A) Multitasking

Multitasking enables scheduling and switching of the CPU between tasks to utilize it to maximum potential. This way, several tasks can run concurrently seemingly even with single core processors. Pipelining architecture further enhances multitasking in processors by enhancing the speed of executions. This is because their execution is critical in real-time systems, where timing constraints are essential.

B) Context Switching

The main process operating in multitasking systems is context switching. The kernel wants to switch between two tasks and stores the context saved in its stack; then, it restores the next task context. It restores a running task from its previous state and then continues to run as if no change occurred. Therefore, the process enables easy, efficient, and timely management of tasks.

C) Order tasks

Every task has an attached priority, which reflects the relative importance of the task within the system. The priorities are usually static for the lifetime of the task or dynamic priority, which may change at runtime based on the needs of the system. Proper assignment of priorities is vital in the RTOS to enable tasks with higher priority get the required CPU time, especially in preemptive systems.

D) Kernel

The kernel is the heart of an RTOS: it performs task management, scheduling, and context switching. It consumes code space in ROM as well as data structure space in RAM, which causes system overhead. Ideally, the overhead of the kernel should take only 2% to 5% of the CPU time so as to be efficient. Kernels supply essential services like semaphore management, mailboxes, queues, as well as time delays; kernels may function in either preemptive or non-preemptive modes.

E) Resource Management

Resources are any resources that tasks need such as input/output devices, variables, structures, or registers. Any resource shared by multiple tasks needs to be managed in order to avoid conflicts and data corruption that may result from the access-exclusion phenomenon, mutual exclusion. A number of techniques are devised for mutual exclusion, including the disabling of interrupts, using "test and set" operations, disabling a program's scheduling, and semaphores to manage the access to common resources.

F) Inter-Task Communication

Tasks or interrupt service routines in real-time systems sometimes must communicate with each other. This can be achieved using global data with mutual exclusion to avoid conflicts. Alternatively, inter-task communication can be achieved through the sending of messages utilizing mechanisms like message mailboxes or message queues.

G) Interrupts

Interrupts allow the CPU to respond immediately to events. In an RTOS, when an interrupt occurs, the ISR will process the event, then return control to the interrupted task (in a non-preemptive kernel) or relinquish control to the highest-priority task running in the system (in a preemptive kernel). Interrupts increase responsiveness because tasks can immediately respond to events outside the task.

H) Clock Tick

A clock tick is a special periodic interrupt that is typically generated in microsecond intervals. It serves as the system's "heartbeat," causing timeouts and allowing tasks to wait for events based on time. The

tick rate affects system overhead; the faster the tick rate, the higher the overhead but with better timing accuracy. Most kernels allow tasks to delay for a specified number of clock ticks, which can be used to support the precise timing control required by real-time application.

III. CHARACTERISTIC

A "real-time system" is sensitive to timing, but it's not an all-or-nothing scenario: certain systems, like those managing mixing or heating, are content with average timing goals that have low variance. Some have rigid deadlines that must be ensured under all conditions—these essential systems of radiation dosing and a few others necessitate the hard-time guarantee. Laplante's definition of a real-time system captures the salient points. "A real-time system is one whose correctness involves both the logical correctness of the outputs and their timeliness[2]. An RTOS is an operating system designed for applications that need adhere to time limits while delivering results that are logically sound (for instance, to ensure that blending takes place prior to and subsequent to heating). RTOSs additionally, offer the essential capabilities to facilitate real-time applications. They offer a fixed environment where we can determine, in advance, the response time for the worst-case scenario.

In laboratory automation, a Real-Time Operating System (RTOS) should provide vital services to guarantee accurate timing, control, and synchronization among different processes. Here's an easy explanation of these essential RTOS services:

A) Asynchronous I/O: Allows the system to keep processing while awaiting the completion of device operations, enhancing efficiency. For instance, the RTOS can start a mixing operation and proceed to a heating request without needing to finish the first task.

B) Synchronous I/O: Enables a thread to halt and wait until an I/O task is finished. This is beneficial when only a single process can manage a device simultaneously, as in the case of sharing a robotic arm across different tasks.

C) Memory Locking: Guarantees that essential process memory remains in physical memory, preventing delays caused by shifting memory to disk. Beneficial

for ensuring that high-priority discussions, such as those tracking essential processes, remain continuously active.

D) Semaphores: Offer methods to coordinate access to shared resources, stopping multiple threads from conflicting by accessing the same resource at the same time.

E) Shared Memory: Enables processes to exchange data by designating a mutual memory space that is accessible to all. Beneficial for procedures that require efficient information exchange.

F) Task Scheduling: Oversees the sequence and importance of tasks. Round-robin scheduling is ideal for sequential low-priority tasks, whereas priority-based preemptive scheduling tackles urgent critical tasks (such as handling overheating).

G) Timers: Offer timing functionalities for planning events at designated intervals, crucial for ensuring real-time management.

H) Interprocess Communication (IPC): Allows threads to share necessary data for the application, utilizing methods such as mailboxes, shared memory, and queues.

I) Real-Time Files: Enables files to be accessed or altered with reliable performance, crucial for data recording in real-time.

J) Real-Time Threads: These are autonomous threads with particular timing needs, essential for guaranteeing that various tasks adhere to their deadlines within a synchronized system.

K) Minimal Overhead: The transition between threads and other operating system tasks should occur swiftly and utilize fewer system resources.

L) Preemptive Scheduling: The operating system can interrupt and prioritize urgent tasks before resuming the execution of lower-priority tasks. For instance, an alarm for system overheating must not allow a currently running less-critical task to proceed with this operation.

M) Deterministic Synchronization: This guarantees

that interactions between any two threads occur within established time limits or frame-locking, making it essential for coordinating intricate laboratory procedures.

N) Priority Levels: Allows for various priority configurations to facilitate adaptable scheduling, ensuring that critical tasks can consistently take precedence over lesser ones.

O) Predefined Latencies: Enables predictable timing for system calls, ensuring that process actions such as mixing or heating can dependably occur in sequence within established time frames.



IV. MISCONCEPTIONS REGARDING RTOS and THEIR CLARIFICATION

Another misunderstanding is that they are inherently "fast"—though it is their determinism, or consistent behavior, that enables them to effectively satisfy those timing constraints, even if not precisely fast due to processing speed, in scenarios where adhering to a deadline is essential.

Not every RTOS is the same. Their characteristics, structures, and appropriateness for various types of real-time applications differ significantly. Optimal solutions have been developed for demanding real-time needs with rigid deadlines, whereas alternatives are more fitting for soft real-time tasks where occasional delays may be permissible. Once more, it relies on the kind of application that necessitates adjustments for its particular timing and resource requirements.

The statement "There is no science in RTOS system design" is incorrect. The design of an RTOS is inherently scientific because many of the fundamental concepts were created and articulated for real-world issues found in embedded systems. This causes methods like deterministic scheduling, priority management, and synchronization to depend on core principles of computer science and fundamental engineering ideas.

An RTOS, by its nature, does not use a significant amount of energy. The energy consumption of an RTOS largely relies on the fundamental hardware and architectural selections. Effective design of an RTOS manages power usage by efficiently using low power states and potentially enhancing task scheduling, allowing even energy-sensitive applications to benefit from it.

RTOS uses considerable amount of CPU overhead. This is not true. Only 1%-4% of CPU time is required by an RTOS[3].

V. LIMITATIONS

Real-time operating systems offer various benefits for time-sensitive applications but also come with specific drawbacks. Here are several key disadvantages of RTOS:

It possesses only a restricted type of multitasking capability. While RTOS is exceptional for managing time-sensitive tasks, it may encounter highly intricate multitasking scenarios if not adequately managed on lower-end hardware.

A) Intricate Evolution: Creating an RTOS is difficult because tasks have strict timing requirements, and the design must be approached with added caution. There exists sophisticated expertise concerning real-time scheduling, priority management, and synchronization-related needs, among other factors, which increase the intricacy of development and associated expenses.

B) Restricted Flexibility: RTOSs lack significant flexibility due to their determinism concerning non-time-critical tasks or unpredictable workloads in their timing demands. They prioritize predictability instead of flexibility, which restricts their applicability for

general-purpose computing.

C) Increased Expenses for Hard Real-Time Needs:

Meeting hard real-time requirements tends to be pricier, necessitating specialized, more costly hardware and extensive testing and validation, which raises the overall project expenses.

D) Scalability Issues: Many RTOSs are greatly reliant on specific applications and may struggle to scale effectively to larger, more complex systems or different hardware environments without considerable redesigns.

E) Energy Usage: Besides the hardware and architecture, the energy usage of an RTOS is influenced by the overhead from continuous task switching, polling, or regular interrupts in the real-time system, contingent on effective management control.

Real-time operating systems offer various benefits for time-sensitive applications but also come with specific drawbacks.

F) Restricted Memory Management: The majority of RTOSs feature restricted memory management since the ability to engage in intricate memory management leads to increased latency. This constraint renders them less appropriate for large-scale dynamic memory allocation tasks.

G) Priority Inversion: In priority-based RTOSs, lower-priority tasks can occasionally delay the execution of higher-priority tasks. This issue can be alleviated through priority inheritance, but it also increases the complexity.

➤ Some Prominent RTOS:-

Name	License	Platforms
FreeRTOS	Modified GNU GPL	ARM, AVR, AVR32, ColdFire, HCS12, IA-32, Cortex-M3, MicroBlaze, MSP430, PIC, PIC32, Renesas H8/S, 8052, STM32, EFM32
LynxOS	Proprietary	Motorola 68010, x86/IA-32, ARM, Freescale PowerPC, PowerPC 970, LEON

RTLinux	GNU GPL	Same as Linux
TI-RTOS	BSD license	Primarily Texas Instruments: MSP430, MSP432, C2000, C5000, C6000, TI's ARM families (Cortex M3/4F, Cortex R4, Cortex A8, Cortex A15), SimpleLink Wireless MCUs (CC2xxx, CC3xxx)
ThreadX	Proprietary	ARC, ARM/Thumb, AVR32, BlackFin, 680x0-ColdFire, H8-300H, Luminary Micro Stellaris, M-CORE, MicroBlaze, PIC24-dsPIC, PIC32, MIPS, V8xx, Nios II, PowerPC, SH, SHARC, StarCore, STM32, StrongARM, TMS320C54x, TMS320C6x, x86/x386, XScale, Xtensa/Diamond, ZSP
VRTX	Proprietary	ARM, MIPS, PowerPC, RISC
VxWorks	Proprietary	ARM, IA-32, Intel 64, MIPS, PowerPC, SH-4, StrongARM, xScale
Windows CE	Proprietary	x86, MIPS, ARM, SuperH

VI. CASE STUDY

ADAS - Enhancing Safety and Efficiency

Advanced Driver Assistance Systems utilize real-time operating systems to ensure smooth and safe operation. Therefore, this case study conducts an assessment of RTOS during its suitable development and use in the ADAS field.

A) A company specializing in embedded systems, Promwad, has effectively adapted the RTOS Zephyr for several ADAS applications. Discussing positively about a company's utilization of RTOS in ADAS by guaranteeing that it upholds these characteristics:

1. Reliable Performance: Zephyr demonstrates predictable behaviour, ensuring that essential ADAS functions operate exclusively within stringent timing

limitations.

2. *Scalability*: Zephyr's lightweight design allows for effective resource utilization when incorporating multiple features of ADAS.

3. *Safety Compliance*: The safety attributes and certifications like IEC 61508 enhance the overall system safety in Zephyr.

4. *Flexibility*: Zephyr's modular design allows for seamless integration of various ADAS sensors and actuators.

B) Obstacles and Factors to Consider:

Although RTOS enhances the performance of ADAS, developers encounter some challenges.

1. *Complexity*: The implementation and optimization of RTOS for ADAS is inherently complex, therefore it necessitates specialized expertise.

2. *Certification*: This adds additional complexity as it requires compliance with automotive safety standards such as ISO 26262.

3. *Resource Limitation*: ADAS systems are generally created to operate in environments with limited resources, necessitating suitable adjustments of RTOS.

C) Future Outlook

As ADAS continues to evolve towards autonomous driving, the role of RTOS will become even more critical. Future developments may include:

1. Increased integration of AI and machine learning algorithms into RTOS for enhanced decision-making[4].
2. Further miniaturization of RTOS to accommodate smaller form factors in next-generation vehicles[4].
3. Enhanced cybersecurity features to protect against potential threats in connected ADAS systems[4].

Real time operating system case study on NASA Mars Curiosity Rover, 2012

Curiosity, which was launched in November 2011, is

fundamentally the Mars Science Laboratory (MSL) rover that touched down on Mars in August 2012. The aim was to investigate Gale Crater and determine if Mars could have supported life at any time in its past.

The VxWorks real-time operating system from Wind River is essential for the Curiosity Rover mission 2. The system allowed the rover to carry out intricate tasks during the Mars landing and subsequent exploration.

A) Essential aspects of VxWorks' function:

1. The rover's avionics incorporated VxWorks, which facilitated an accurately intricate landing.
2. The RTOS enabled the rover to respond to sensor data in real time with millisecond accuracy from sensors throughout the landing procedure.
3. VxWorks has been utilized in various critical space missions, including the Juno mission to Jupiter and the Spitzer Space Telescope.

B) Why is an RTOS necessary for a Space Mission?

Real-time operating systems such as VxWorks are essential to a spacecraft for multiple reasons:

1. Specifically, they enable systems to react with remarkable precision to live sensor data .
2. *Synchronization*: RTOS facilitates efficient communication among various components of the spacecraft and with ground control.
3. *Safety*: The safety and reliability of RTOS are essential for mission-critical tasks, like landing on planetary surfaces.
4. *Resource distribution*: RTOS effectively manages hardware resources, allowing complex systems to operate efficiently under stringent constraints.

C) Difficulties in Mars Exploration

The Curiosity Rover faced these difficulties during its mission:

1. *Entry, Descent, and Landing EDL*: The rover faced a dangerous entry into Mars' sparse atmosphere, encountering temperatures of nearly 3000°C.

2. *Communication lag*: Signals traveling to and from Mars require about 20 minutes for each journey, resulting in careful planning of operations.

3. *Severe environment*: The rover faces extreme temperature fluctuations and radiation levels on the surface of Mars.

D) Impact of RTOS on Mission Success

The use of VxWorks RTOS was crucial for several aspects of the Curiosity Rover mission:

1. *Landing precision*: The RTOS enabled precise control during the EDL phase, allowing the rover to land safely on Mars' surface.[5]
2. *Scientific operations*: After landing, VxWorks continued to power the rover's complex scientific instruments and systems.[5]
3. *Long-term reliability*: The RTOS played a role in maintaining the rover's functionality over extended periods, including through dust storms and equipment failures.[5]

Tesla Autopilot System - RTOS: Custom RTOS

Tesla's Autopilot system facilitates the car's advanced driver-assistance features using a proprietary real-time operating system. Since Tesla's RTOS is proprietary, we are unable to provide extensive details about it. We can draw certain conclusions based on Tesla's requirements and the system's performance.

A) Essential Characteristics of Tesla's RTOS

1. *In-house Development*: Tesla likely created its own proprietary RTOS tailored for applications specific to the automotive industry.

2. *Real-Time Performance*: An RTOS needs to deliver extremely low-latency responses to guarantee the safety of autonomous driving operations.

3. *Supports Multicore*: The computational requirements of Autopilot were likely met by necessitating multi-core processing support from the RTOS.

4. *Safety-Critical Design*: The system must adhere to rigorous safety standards relevant to the automotive industry.

B) Obstacles to Development

1. *Complexity*: Creating an RTOS for an autonomous vehicle's driving system requires significant complexity along with enhanced reliability.

2. *Balancing Performance and Safety*: Addressing both performance and strong safety features within the limits of real-time operation presents a challenging issue to be addressed.

3. *Scalability*: As the features of Autopilot begin to expand, the scalability of the RTOS must also match the increasing computational demands.

C) NHTSA Investigation Findings[6]

1. *Fatal Collision Incident*:

- The investigation follows a fatal collision involving a Tesla driver using FSD.
- The incident occurred when the driver was using FSD within 30 seconds of the crash.

2. *Reduced Roadway Visibility Conditions*:

- The probe will evaluate whether Tesla's FSD systems are safe to use in conditions such as fog, glaring sun, or other reduced roadway visibility scenarios.

3. *Number of Incidents*:

- As of October 1, 2024, NHTSA had tracked 1,399 incidents involving Tesla's driver assistance systems being engaged within 30 seconds of a collision.
- Of these, 31 incidents resulted in fatalities.

4. *Vehicle Population Affected*:

- The investigation pertains to approximately 2.4 million Tesla electric vehicles on U.S. roads, including:
 - Model S and X vehicles produced from 2016 to 2024

- Model 3 vehicles produced from 2017 to 2024
- Model Y vehicles produced from 2020 to 2024
- Cybertruck vehicles produced in 2023 and 2024.

5. Software Updates:

- The investigation will examine Tesla's over-the-air software updates to its FSD systems.
- NHTSA aims to understand the timing, purpose, and capabilities of these updates, as well as Tesla's assessment of their safety impact.

RTOS in Medical Devices – Pacemaker

A prominent medical device firm was developing a sophisticated pacemaker system where precision and timing were crucial in every aspect. The new product needed to deliver top performances in a trustworthy range of high-safety items.

A) Obstacles

1. The pacemaker needs accurate and prompt pacing signals.
2. Rapid reaction to physiological changes.
3. Safety-critical operations guaranteeing reliability while maintaining patient safety.
4. Memory needed so as to securely retain essential software and patient information.

Solution: Deployment of RTOS

They resolved to adopt a real-time operating system (RTOS) to curb the challenges outlined above. Ultimately, the company settled on SAFE RTOS as the operating system. This is because it contained many safety-critical features and was targeted for compliance with medical device regulation.

In this context, the producer embraced the challenge with the idea of a real-time operating system (RTOS). It selected SAFE RTOS, known for its safety-critical features and compliance with medical device standards.

B) Main Characteristics

1. *Hardware:* Customized microcontroller with low

power usage.

2. *Software:* SAFE RTOS equipped with integrated safety functionalities.

3. *Memory:* Protected EEPROM designed for saving essential software and patient information.

4. *Execution Information*

- The preemptive scheduling system guarantees the timely execution of essential tasks.
- Rate monotonic analysis is utilized for deterministic task scheduling.

5. *Safety Attributes:*

- Integrated error detection and correction systems.
- Adherence to ISO 14971 standards for risk management.

6. *Memory Safeguarding:*

- Call tables facilitate a secure connection with software that can be downloaded.
- Non-volatile memory (EEPROM) for safeguarding and preserving essential software

6. *Power Management:*

Efficient resource distribution for prolonged battery duration.

C) Results[7]

1. Improved Precision:

- RTOS enables precise timing control for pacing signals, reducing jitter and improving treatment efficacy.

2. *Enhanced Reliability:*

- The real-time capabilities of RTOS ensure consistent operation and robust error handling mechanisms.
- This leads to improved reliability and reduced risk of device failure.

3. *Faster Response Times:*

- Low-latency interrupt handling allows for quick response to physiological changes.
- This enables faster reaction times in delivering electrical impulses when needed.

4. *Deterministic Behaviour:*

- RTOS provides predictable response times, which is crucial for medical devices that require precise control over critical functions.
5. *Safety Compliance:*
- RTOS solutions like SAFE RTOS comply with safety-critical design standards and regulations.
 - This ensures that the pacemaker meets rigorous safety requirements set by organizations like the FDA.
6. *Extended Battery Life:*
- Optimized power management features in RTOS help extend the battery life of implantable devices like pacemakers.
7. *Regulatory Approval:*
- RTOS-based pacemaker systems can meet regulatory requirements such as FDA 510(k) class III clearance and IEC 62304 class C certifications.

VII. CONCLUSION

For many years, RTOSs have been mainly used in embedded systems. However, in recent times, there has been a rise in the creation of applications for regular platforms that require real-time capabilities. With the growing demand for real-time systems, it's important to be aware of the advantages and drawbacks of RTOSs in order to make an informed decision when necessary. The information presented will offer insight to assess the technical aspects and non-technical factors of RTOS. Part of the selection process is to determine real-time constraints and classify them as hard, firm, or soft. Of course, if there are no hard time constraints, then probably a current OS would be fine. But for a hard or firm real-time system, you'll need to weigh the trade-offs between the different RTOSs and decide which one fits your needs and budget. Then consider the ownership cost of the RTOS, the hardware platform, the development tools, and most importantly whether the RTOS adheres to the deadline constraints. Not all criteria used to evaluate should be given the same weights, thus make sure to emphasize the most important features to the system. In case all the criteria are put into consideration appropriately, then the chosen RTOS is the correct one for then. And then this investment could eventually pay for itself over the years.

REFERENCES

- [1]Stankovic, John A., and Raj Rajkumar. "Real-time operating systems." *Real-Time Systems* 28.2-3 (2004): 237-253.
- [2]Laplante, P. A. *Real Time Systems Design and Analysis*. 3rd edition Hoboken, NJ: Wiley; 2004; xxi, 505 p.
- [3]Deadlock[Online]. Available: en.wikipedia.org/wiki/Deadlock
- [4]Moroshek, P. (2004, November 5). RTOS in ADAS: Enhancing safety and efficiency.
- [5] Dannels, Deanna Patricia. Abstract. "Orality in the Disciplines: A Study of Oral Practices in Mechanical Engineering." *University Microfilms No. AAT 9924003*. U of Utah, 1999. Academic.writing.2000.<http://wac.colostate.edu/aw/theses/index.htm> (10 April 2000).
- [6]"Tesla faces NHTSA investigation of 'Full Self-Driving' after fatal collision", CNBC, October 18, 2024.
- [7] BasuMallick , C. (2024, March 7). What is RTOS? Spiceworks.Retrieved from <https://www.spiceworks.com/tech/hardware/articles/what-is-rtos/>